

Time and Space Complexity (Big-O)

What Big-O Notation Measures

Big-O notation describes how the runtime or memory usage of an algorithm grows as the input size (n) grows, focusing on the worst-case scenario and ignoring constant factors. It answers: 'if I double the input, roughly how much slower does this get?' This is the language interviewers use to evaluate whether your solution is 'optimal enough'.

Common Complexities, Fastest to Slowest

- $O(1)$ — Constant: runtime doesn't change with input size (e.g. array index access, hash map lookup).
- $O(\log n)$ — Logarithmic: runtime grows slowly as input grows (e.g. binary search, balanced BST operations).
- $O(n)$ — Linear: runtime grows proportionally with input size (e.g. a single loop through an array).
- $O(n \log n)$ — Linearithmic: typical of efficient sorting algorithms (merge sort, quicksort average case).
- $O(n^2)$ — Quadratic: typical of nested loops over the same input (e.g. bubble sort, naive pair-checking).
- $O(2^n)$ — Exponential: typical of naive recursive solutions that branch repeatedly (e.g. naive Fibonacci, brute-force subsets).
- $O(n!)$ — Factorial: typical of brute-force permutation generation.

How to Estimate Complexity by Reading Code

- A single loop through n elements: $O(n)$.
- A loop nested inside another loop, both over n : $O(n^2)$.
- A loop that halves the search space each time (like binary search): $O(\log n)$.
- A recursive function that makes two calls on $n-1$ each time with no memoization: $O(2^n)$.
- Sorting, then a single pass: $O(n \log n)$ (dominated by the sort).

Space Complexity

Space complexity measures the extra memory an algorithm uses relative to input size, separate from the space taken by the input itself. Recursive functions have $O(\text{depth of recursion})$ space due to the call stack, even if they don't create new data structures — this is a common trap in interviews when candidates only count explicit variables.

Why This Matters for Interviews

After solving any problem, interviewers almost always ask 'what's the time and space complexity of your solution, and can you do better?' Being unable to state this clearly is one of the most common reasons otherwise-correct solutions lose points. Always state Big-O immediately after presenting a solution,

without being asked.